

Fecha: 28 de Septiembre del 2026

CNO 4: CIBERSEGURIDAD

ACTIVIDAD 12 LAB: VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

Documentación

Maestro: Servando López Contreras

Alumno: Quetzal Olin Rodríguez Jiménez

Contenido

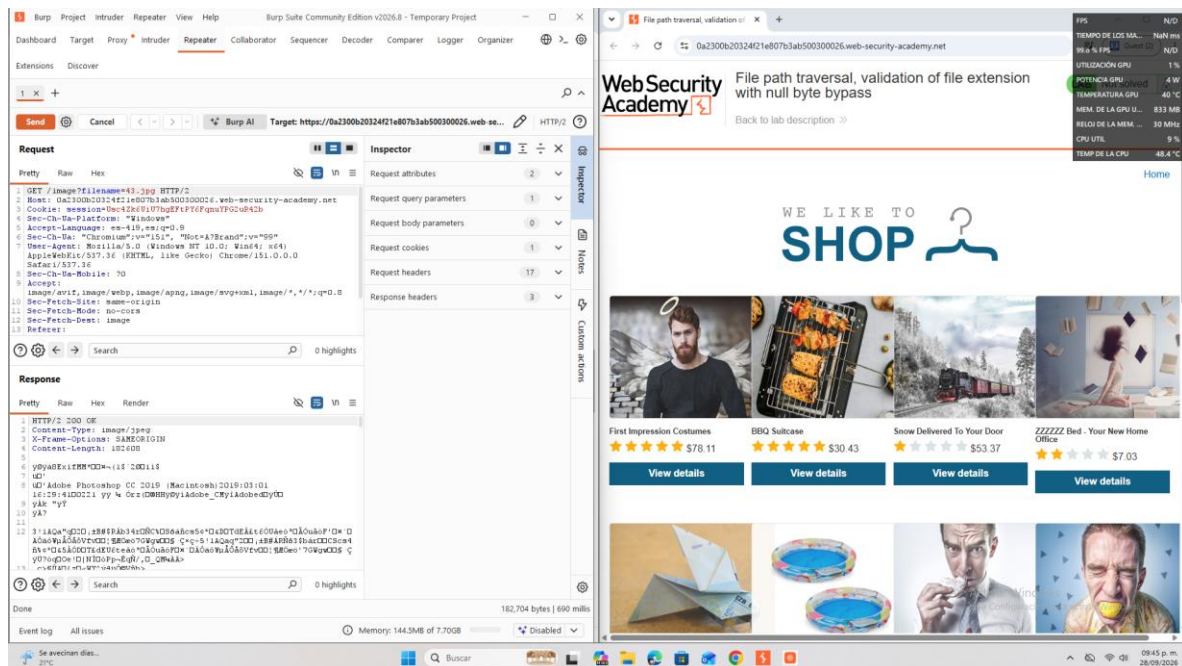
OBJETIVO.....	2
AMBEINTE DE LABORATORIO	3
ANÁLISIS DE LA VULNERABILIDAD Y FORMA DE RESOLUCIÓN.....	4
COMANDOS UTILIZADOS	5
RESULTADOS	5
CONCLUSIONES	6

OBJETIVO

Explotar una vulnerabilidad de Path Traversal (Salto de Directorio) donde el servidor valida estrictamente que el nombre del archivo solicitado termine con una extensión específica y permitida (por ejemplo, .png o .jpg). El objetivo es evadir esta restricción inyectando un byte nulo (%00) antes de la extensión obligatoria, lo que engaña a la lógica de validación pero provoca que la API del sistema de archivos backend trunque la cadena y lea un archivo fuera del directorio (como /etc/passwd).

AMBEINTE DE LABORATORIO

1. Captura de la petición objetivo Localiza la petición GET que carga los recursos estáticos y que incluye el parámetro filename= (por ejemplo, al cargar la imagen de un producto). Envía esta petición a la herramienta Repeater. *[Insertar Imagen 1: Captura de la petición original en Burp Suite Proxy/Repeater mostrando la solicitud de una imagen válida, ej. filename=72.png]*



2. Confirmar la validación de la extensión Prueba realizar un ataque de path traversal estándar modificando el parámetro (ej. filename=.././../etc/passwd). Observa la respuesta del servidor, la cual probablemente arroje un error o rechace la solicitud indicando que el archivo "debe terminar con una extensión permitida como .png". *[Insertar Imagen 2: Respuesta del servidor rechazando el payload estándar de path traversal por no cumplir con la validación de la extensión]*

The screenshot displays the Burp Suite interface on the left and the Web Security Academy challenge page on the right. In Burp Suite, the 'Repeater' tab is active, showing a request to `https://0a2300b20324f21e807b3ab500300026.web-security-academy.net`. The request is a GET for `/image?filename=../../../../etc/passwd%00.png`. The response is an HTTP 200 OK with a Content-Type of `application/json` and a Content-Length of 14. The response body is `"No such file"`. The Web Security Academy page shows the challenge title 'File path traversal, validation of file extension with null byte bypass' and a 'We Like To SHOP' banner.

3. Prueba de bypass con inyección de Null Byte (%00) Modifica el parámetro añadiendo el payload de salto de directorio, seguido del archivo objetivo, el byte nulo codificado en URL y, finalmente, la extensión permitida. Ejemplo: `filename=../../../../etc/passwd%00.png` El servidor valida correctamente que la cadena termina en `.png`, pero al interactuar con el sistema operativo, la ruta se corta en `%00`, permitiendo la lectura del archivo. *[Insertar Imagen 3: Petición en Repeater con el payload de null byte exitoso y la respuesta HTTP mostrando el contenido del archivo `/etc/passwd`]*

The screenshot displays the Burp Suite interface on the left and the Web Security Academy challenge page on the right. In Burp Suite, the 'Repeater' tab is active, showing a request to `https://0a2300b20324f21e807b3ab500300026.web-security-academy.net`. The request is a GET for `/image?filename=../../../../etc/passwd%00.png`. The response is an HTTP 200 OK with a Content-Type of `image/png` and a Content-Length of 2316. The response body is a binary image file. The Web Security Academy page shows the challenge title 'File path traversal, validation of file extension with null byte bypass' and a 'We Like To SHOP' banner. A message at the top of the page says 'Congratulations, you solved the lab!'.

ANÁLISIS DE LA VULNERABILIDAD Y FORMA DE RESOLUCIÓN

Discrepancia en el manejo de cadenas (String Handling): La vulnerabilidad ocurre por la diferencia en cómo distintos componentes procesan los textos. El lenguaje de alto nivel (como PHP o Java antiguo) valida toda la cadena asumiendo que termina en `.png`. Sin embargo, las APIs de bajo nivel del sistema operativo (escritas en C/C++) utilizan cadenas terminadas en null (Null-terminated strings). Al encontrar el byte nulo (`%00`), el sistema operativo asume que es el final del nombre del archivo, ignorando el `.png` que viene después.

Validación de sufijo engañada: Funciones como `endsWith(".png")` devuelven verdadero (True) porque evalúan la cadena completa tal como se envía en la petición web, sin considerar cómo la interpretará el sistema de archivos.

Forma de resolución:

- **Actualización del stack tecnológico:** En versiones modernas de PHP ($\geq 5.3.4$) y Java ($\geq 7u40$), los lenguajes lanzan una excepción si detectan un byte nulo en la ruta de un archivo, mitigando este ataque por defecto.
- **Sanitización estricta:** Rechazar cualquier solicitud que contenga caracteres anómalos o de control (como `%00`) antes de cualquier otra validación.
- **Evitar referencias directas:** Utilizar identificadores (como hashes o IDs de base de datos) en lugar de permitir que los usuarios controlen directamente los nombres de los archivos físicos.

COMANDOS UTILIZADOS

Burp Suite – Proxy (Intercept ON): Se activó la opción *Intercept on* para capturar las peticiones HTTP generadas por el navegador.

Forward: Se utilizó el botón *Forward* para permitir que la página cargara después de interceptar las solicitudes.

Observación de tráfico GET: Se identificaron múltiples peticiones de tipo **GET** hacia recursos estáticos.

Prueba de acceso a directorios: Se intentó navegar hacia el directorio raíz regresando niveles (. . /) para comprobar la estructura de directorios y posibles accesos indebidos.

Send to Repeater: Envío de la solicitud del parámetro filename al entorno de pruebas repetitivas de Burp Suite.

Manipulación de Parámetro GET (filename): Modificación del valor por secuencias anidadas no recursivas (....//) para evadir los filtros de validación de entradas.

RESULTADOS

Confirmación de la vulnerabilidad: Se comprobó que el servidor realiza una validación superficial de la extensión del archivo, la cual puede ser evadida exitosamente utilizando la técnica de inyección de byte nulo.

Payload efectivo: El uso del payload . . / . . / . . / etc / passwd%00 . png fue procesado correctamente por la aplicación y el sistema subyacente. La validación web aceptó el sufijo . png, mientras que el sistema operativo leyó únicamente la ruta hasta passwd.

Comportamiento del servidor: El servidor devolvió exitosamente el contenido del archivo solicitado (/etc/passwd), demostrando el acceso no autorizado a los directorios internos del servidor.

CONCLUSIONES

Confidencialidad: Existe un impacto crítico en la confidencialidad. Se demostró la capacidad de extraer archivos sensibles del sistema operativo. Si la aplicación alberga código fuente, archivos de configuración o credenciales (como un archivo . env), estos también podrían ser leídos.

Escalada de riesgo: Un atacante puede utilizar esta técnica para realizar un reconocimiento interno exhaustivo (enumeración de usuarios a través de passwd, lectura de claves privadas, etc.), lo que facilita vectores de ataque más graves, como la ejecución remota de código (RCE).

Alcance: Este vector de ataque afecta a sistemas que utilizan lenguajes/frameworks heredados (legacy) que no manejan de forma segura los bytes nulos al interactuar con el sistema de archivos, y que confían ciegamente en extensiones controladas por el usuario.